

Tabla de contenido

Redirecciones y Pipes	1
Redirecciones	1
Bifurcación (comando tee)	2
Tuberías o pipes	2
Actividades.....	3

Redirecciones y Pipes

Cuando utilizamos la consola a menudo la salida de un comando la tenemos que aprovechar en otro, preferiríamos que la salida se nos guardase directamente en un fichero, o simplemente deseamos utilizar cierta información de la salida de dicho comando. En GNU/Linux, hay dos mecanismos que nos facilitan enormemente esta tarea, y que con la costumbre llegan a ser casi imprescindibles: las redirecciones, y los *pipes*(o tuberías).

Redirecciones

En GNU/Linux, al final todo es tratado como si fuera un fichero y como tal, tenemos **descriptores de fichero** para aquellos puntos donde queramos acceder. Hay unos descriptores de fichero por defecto:

- **0**: Entrada estándar /dev/stdin (normalmente el teclado).
- **1**: Salida estándar /dev/stdout (normalmente la consola).
- **2**: Salida de error /dev/stderr.

Para redirigir las salidas utilizaremos el **descriptor de fichero** seguido del símbolo '>' (o < si redirigimos la entrada hacia un comando). Veamos unos ejemplos:

```
$ ls -l >fichero
```

Guarda la salida de *ls -l* en *fichero*. Si no existe lo crea, y si existe lo sobrescribe.

```
$ ls -l >>fichero
```

Añade la salida del comando a *fichero*. Si no existe lo crea, y si existe, lo **añade al final**.

```
$ ls -l 2>fichero
```

Si hay algún error, lo guarda en *fichero* (podría salir un error si no tuviéramos permiso de lectura en el directorio).

Es importante ver que **si no se especifica el descriptor de fichero se asume que se redirige la salida estándar**. En el caso del operador < se redirige la entrada estándar, es decir, el contenido del fichero que especificáramos, se pararía como parámetro al comando.

Si quisiéramos redirigir todas las salidas a la vez hacía un mismo fichero, podríamos utilizar **>&**. Además, con el carácter **&** podemos redirigir salidas de un tipo hacia otras, por ejemplo, si quisiéramos redirigir la salida de error hacia la salida estándar podríamos indicarlo con: **2>&1**. Es importante tener en cuenta que el orden de las redirecciones es significativo: se ejecutarán de izquierda a derecha.

Bifurcación (comando tee)

A veces interesa que la salida de un comando, además de redirigirse a un determinado fichero, se bifurque también hacia la terminal, con objeto de observar inmediatamente el resultado. Esto se consigue con el operador tee, que podría emplearse de la siguiente forma:

```
ls -a | tee archivo1
```

la salida de ls se bifurca hacia la terminal y hacia archivo1.

Si quisiéramos que la salida de este comando se añadiera al final de archivo1, deberíamos utilizar la opción **-a** (del comando tee)

```
ls -a | tee -a archivo1
```

Tuberías o pipes

Este mecanismo nos permite pasar la salida de un comando a otro. Para ello se usa la sintaxis: **<comando1> | <comando2>**. Con esto, la salida de *comando1* será la entrada de *comando2*. Vamos a ver unos ejemplos:

```
$ rpm -qa | grep <nombre_paquete>
```

El primero de los dos comandos nos haría una lista de todos los paquetes instalados. Imaginemos que sólo queremos saber si tenemos instalado uno en concreto. Con el segundo comando limitamos la salida a los paquetes que en el nombre que contengan el *patrón* que especificamos en *<nombre_paquete>*. Por ejemplo, para saber si tenemos instalado algún paquete llamado **glibc** haríamos:

```
$ rpm -qa | grep "glibc"
```

grep es un *parseador* de expresiones regulares, es decir, le damos un patrón y un fichero (o introducimos lo que sea por consola, o lo pasamos con un *pipe*) y de ese texto nos devuelve sólo lo que coincide con el patrón. Este es el funcionamiento básico de **grep** pero es muchísimo más potente. Para más información: `man grep`.

Otro ejemplo útil sería, por ejemplo, cuando queremos saber el PID de un [proceso](#). En vez de mostrarlos todos y tener que buscarlo podríamos hacer:

```
$ ps -e | grep <nombre_proceso>
```

y así nos mostraría sólo las líneas que contuvieran *<nombre_proceso>* (es decir, limitaríamos la salida al proceso que queremos).

Carácter	Resultado
comando < fichero	Toma la entrada de fichero
comando > fichero	Envía la salida de comando a fichero ; sobrescribe cualquier cosa de fichero
comando 2> fichero	Envía la salida de error de comando a fichero (el 2 puede ser reemplazado por otro descriptor de fichero)
comando >> fichero	Añade la salida de comando al final de fichero
comando << etiqueta	Toma la entrada para comando de las siguientes líneas, hasta una línea que tiene sólo etiqueta
comando 2>&1	Envía la salida de error a la salida estándar (el 1 y el 2 pueden ser reemplazado por otro descriptor de fichero, p.e. 1>&2)
comando &> fichero	Envía la salida estándar y de error a fichero ; equivale a comando > fichero 2>&1
comando1 comando2	pasa la salida de comando1 a la entrada de comando2 (<i>pipe</i>)

Actividades

- Utiliza redirecciones para realizar los siguientes ejercicios
 - Crea el fichero llamado **ejemplo** conteniendo la salida de **ls -l**
 - Añade a **ejemplo** el contenido del directorio **/etc**
 - Muestra el contenido de **ejemplo** página a página (**more**)
 - Pon un ejemplo donde se envía los mensajes de error al dispositivo nulo (a la basura **/dev/null**)
 - Crea el fichero **pimienta** vacío
 - Guarda toda la información que se introduzca por teclado hasta que se pulse la tecla **Ctrl+D** en el fichero **ejemplo2**
 - Guarda toda la información que se introduzca por teclado hasta que se introduzca la línea que sólo sea la palabra **ADIOS**. Guarda los datos en el fichero **ejemplo3**
 - Realiza un **ls -lR /** con un usuario (no **root**) y guarda la salida en fichero **ejemplo4** y el error en **ejemerr4**
 - Realiza un **ls -lR /** con un usuario (no **root**) y guarda la salida en fichero **todo** tanto la salida como el error
 - El siguiente comando tiene un error lógico. ¿sabría cuál?
ls -l -R / 2>&1 > todo
 - Muestra el contenido de **/etc/group** en otra terminal terminal, es decir, ejecutamos el comando en un sitio y el resultado se muestra en otra terminal (usar el comando **tty** para ver el nombre del terminal)
- ¿Qué hacen los siguientes comandos?
 - ls -lR / | grep group**
 - ls -l / | tee melón**
 - grep -i '10' /etc/passwd | sort | uniq**
 - cat /etc/group | grep ^a**
 - cat /etc/group | grep :\$**